ASSIGNMENT-1: Linear Regression on California Housing Dataset

Machine Learning with Python — Project Report

1. Introduction & Core Libraries

- **NumPy (Numerical Python):** handles matrices, large arrays, and forms the foundation for ML math
- **Pandas:** tables and Excel-like data manipulation
- **Matplotlib / Seaborn:** graphs, plots, and statistical visualisations
- **Scikit-learn:** Machine Learning models and built-in datasets
- **A package** is reusable code written by other programmers
- Install all required packages:

```
pip install pandas numpy scikit-learn matplotlib seaborn jupyter
```

2. Virtual Environment Setup

- **Why needed:** avoids dependency conflicts — different projects need different library versions; installing one version globally can break another
- **Create the environment:** `sudo apt install python3-full python3-venv -y`
- **Activate it:** `source ml_env/bin/activate`
- Does not need to be created each time — just activate to reuse

3. Dataset Overview — California Housing

- **Loaded via** `fetch_california_housing(as_frame=True)` from scikit-learn
- `as_frame=True`**:** returns data as a Pandas DataFrame
- **Shape:** 20,640 rows × 9 columns
- **8 features:** MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude
- **Target variable:** MedHouseVal (Median House Value in $100,000s)
- **No missing values** (verified using `df.isnull().sum()`)

4. Implementation Steps

4.1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

4.2 Load Dataset

```
data = fetch_california_housing(as_frame=True)
df = pd.concat([data.data, data.target.rename('MedHouseVal')], axis=1)
df.head()
```

4.3 Exploratory Data Analysis

```
print(df.shape)
print(df.describe())
print(df.isnull().sum())
```

- `df.shape` → (20640, 9)
- `df.describe()` → statistical measures: mean, std, min, 25%, 50%, 75%, max
- `df.isnull().sum()` → confirms no missing values

4.4 Correlation Heatmap

```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Feature Correlation')
plt.show()
```

- Purpose: visualise feature relationships and identify multicollinearity

4.5 Train–Test Split (80/20)

```
X = df.drop(columns='MedHouseVal')
y = df['MedHouseVal']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f'Training rows: {len(X_train)}, Testing rows: {len(X_test)}')
```

- **Output:** Training rows: 16,512, Testing rows: 4,128
- `random_state=42`**:** ensures reproducibility (same split every run)

4.6 Model Training

```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

4.7 Model Evaluation

```
mae  = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2   = r2_score(y_test, y_pred)
print(f'MAE:  {mae:.3f}')
print(f'RMSE: {rmse:.3f}')
print(f'R2:   {r2:.3f}')
```

5. Evaluation Metrics — Results

Metric	Value	Meaning
MAE	0.533	Average absolute prediction error (~$53,300 off on average)

Metric	Value	Meaning
MSE	0.556	Squared errors averaged — penalises bigger mistakes more heavily
RMSE	0.746	Square root of MSE; same unit as target variable
R^2 Score	0.576	Model explains ~57.6% of variance in housing prices

R^2 Interpretation

- **1.0** → Perfect prediction
- **0** → Model predicts only the average value (no useful information)
- **< 0** → Worse than predicting the average

6. Visualizations

- **Correlation Heatmap —** shows pairwise feature correlations; red = positive, blue = negative
- **Actual vs Predicted Scatter Plot —** points compared against a red y = x line; closer to the line = better prediction
- **Residuals Histogram —** distribution of (actual − predicted) errors; should be roughly normal around 0

```
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.4)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], color='red', linewidth=2)
plt.xlabel('Actual Price'); plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted House Prices')
plt.show()
```

7. Outlier Analysis

- Outliers were identified using `df.describe()` by comparing mean vs maximum values.

Feature	Max Value	Typical (Mean)	Issue
AveRooms	141.91	5.43	Normal homes have 3–10 rooms; values >100 are suspicious
AveBedrms	34.07	1.10	Most houses don't have dozens of bedrooms
AveOccup	1,243.33	3.07	1000+ people per household is unrealistic
Population	35,682	~1,425	Some block groups have extreme populations
MedHouseVal	5.0 (capped)	2.07	Target capped at 5.0; model cannot distinguish expensive houses

- **No missing values** were found in the dataset
- **No outlier removal code** was applied in this assignment (acknowledged limitation)
- **These outliers** contribute to the moderate R^2 score of 0.576

8. Saving the Model (Pickle)

```
import pickle
```

```
with open('house_price_model.pkl', 'wb') as f:
    pickle.dump(model, f)

print('Model saved!')
```

- **Pickle:** converts Python objects into binary .pkl files for later reuse without retraining
- `wb` **=** write binary mode — required because pickle writes raw bytes
- **.pkl extension:** standard convention for pickle files

9. Suggested Improvements

- **Feature Engineering:** Instead of raw room counts, create meaningful ratios such as rooms-per-person. More meaningful inputs lead to better predictions.
- **Feature Scaling:** Population (~1,425 avg) and AveBedrms (~1.1 avg) operate on vastly different scales. Standardisation ensures the model treats all features fairly.
- **Ridge Regression:** AveRooms and AveBedrms are 85% correlated. Ridge adds a penalty to large coefficients, preventing the model from over-relying on near-duplicate features and spreading weight more evenly.
- **Cross-Validation:** A single 80/20 split can get lucky. K-fold cross-validation (e.g. k=5) tests the model on 5 different splits and averages the score for a more reliable accuracy estimate.
- **XGBoost (Extreme Gradient Boosting):** Linear regression draws a straight line — house prices are not linear. XGBoost builds hundreds of small decision trees where each tree corrects the mistakes of the previous one, progressively minimising error. This can push R^2 from 0.576 to 0.83+.

10. Conclusion

- Successfully built a Linear Regression model on the California Housing dataset
- **Achieved** R^2 = 0.576 and RMSE = 0.746 on the test set
- Identified significant outliers in AveRooms, AveBedrms, AveOccup, and a hard cap on MedHouseVal
- Performance can be improved by outlier handling, feature scaling, and trying non-linear models such as Random Forest or Gradient Boosting

---